

Scholar's Tea - Finding Mutual Sparks of Science While Drinking a Cup of Tea

项目愿景

打造高校学术交流社区，在这里，不同高校、不同领域的不同课题组均可入住，交流分享自己的工作、探讨产生新的思想火花。主体功能参考 Github 社区和浙大 cc98(cc98.org) 论坛风格。

一、核心功能模块

1.1 课题组分类（Research Group Hub）

定位：类似 GitHub 的组织（Organization）页面

功能矩阵：

功能	描述	优先级
课题组主页	展示课题组基本信息、logo、导师、成员	P0
信源发布	上传论文、基金申请书、新闻等	P0
AI 总结	大模型总结用户选取的信源内容	P0
互动系统	点赞、评论、收藏	P0
评分权重	优质评论>普通评论>点赞	P1
引用追踪	用户引用某信源的追踪记录	P1
基金申请	课题组用信源申请基金时的 AI 辅助	P2

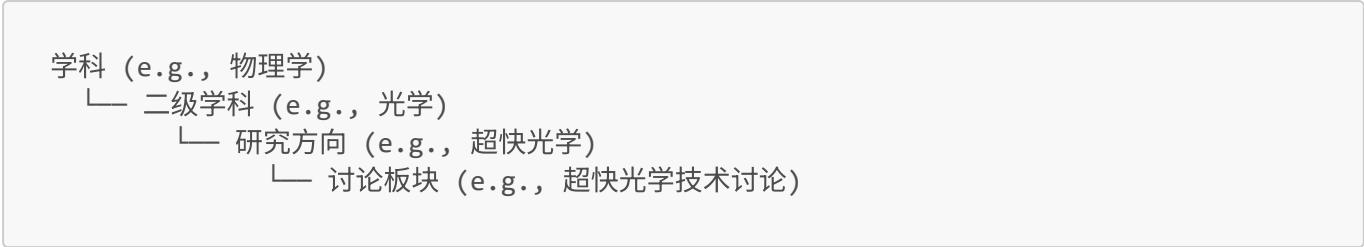
数据结构：

```
ResearchGroup
├── id, name, institution, college, department
├── logo, banner, description
├── advisor_id, leader_id, member_ids[]
├── publications[], news[], patents[]
├── score, rank
└── verification_status
```

1.2 学科社区（Discipline Community）

定位：垂直领域知识图谱 + 论坛

层级结构：



特殊组件：

- 每级页面显示「前十优质课题组」实时榜单
- 跳转链接：用户主页 → 课题组主页（可穿透）
- 跨学科标签系统（一个帖子可打多个方向标签）

1.3 Top Ten Questions

定位：每月热门话题聚合

功能：

- 用户投票评选当月 Top 10 问题
- 问题讨论区（按点赞/时间排序）
- 优质回答高亮 + 引用追踪
- 问题状态追踪（已解决/待解决）

1.4 Tea Party（实时学术聊天室）

定位：类腾讯会议/钉钉的实时交流

功能矩阵：

功能	描述
茶话会房间	创建公开/私密学术讨论房间
文字聊天	实时文本交流
屏幕共享	演示文档、代码、白板
在线用户	显示当前在线的其他用户
历史记录	房间聊天记录可追溯
邀请机制	邀请特定用户加入

技术选型：WebSocket (Socket.io/WS) + 房间管理系统

1.5 思想工坊（AI 助手）

定位：学术专用 AI 对话助手

能力：

- 科学想法讨论与评估
- 解决方案头脑风暴

- 基金申请指导
- 论文撰写辅助（润色、结构优化）
- 参考文献推荐

实现方式：

- 接入 Claude API / 私有模型
- RAG 知识库（领域专业语料）
- 多轮对话上下文管理

1.6 数据获取与冷启动

挑战：如何获取初始课题组的论文、网页等数据

解决方案：

方式	实现	难度
学术 API	Semantic Scholar、Google Scholar API	中
爬虫	爬取高校/科研机构官网	高
用户贡献	课题组自行上传	低
合作数据	与学术数据库合作	极高

二、技术架构

2.1 前端技术栈

层级	技术	理由
框架	Next.js 14+ (App Router)	SSR/SSG、SEO 友好
UI 库	shadcn/ui + Tailwind CSS	快速迭代、风格统一
状态管理	Zustand / Jotai	轻量级
实时通信	Socket.io Client	Tea Party 功能
富文本	Tiptap / Slate	论文、评论编辑
搜索	Algolia / Typesense	全文搜索
AI 集成	LangChain.js	思想工坊

2.2 后端技术栈

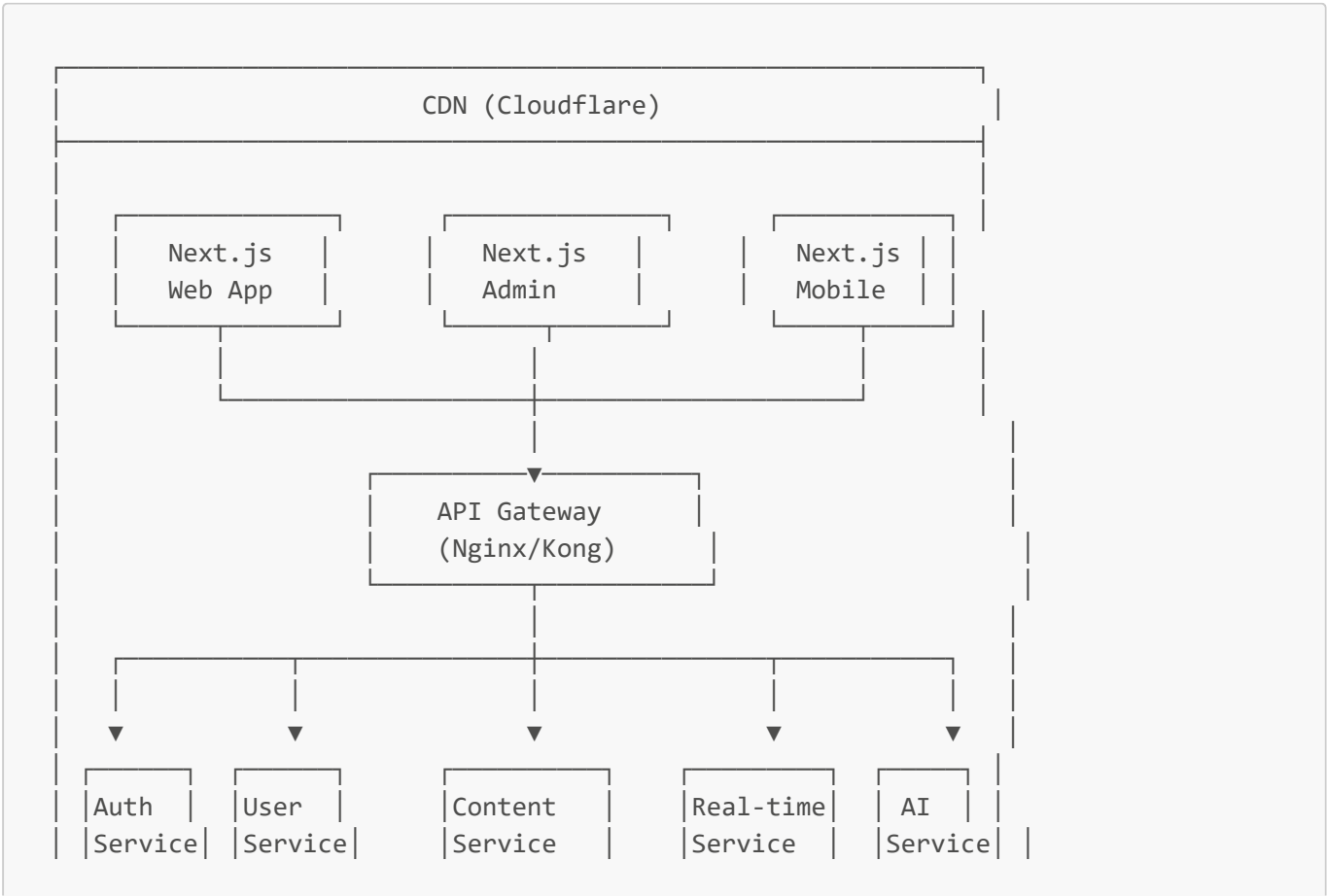
层级	技术	理由
运行时	Node.js 20+ / Bun	高并发、类型安全
框架	Fastify / Hono	高性能、类型推断
数据库	PostgreSQL 16+	结构化数据、JSON 支持

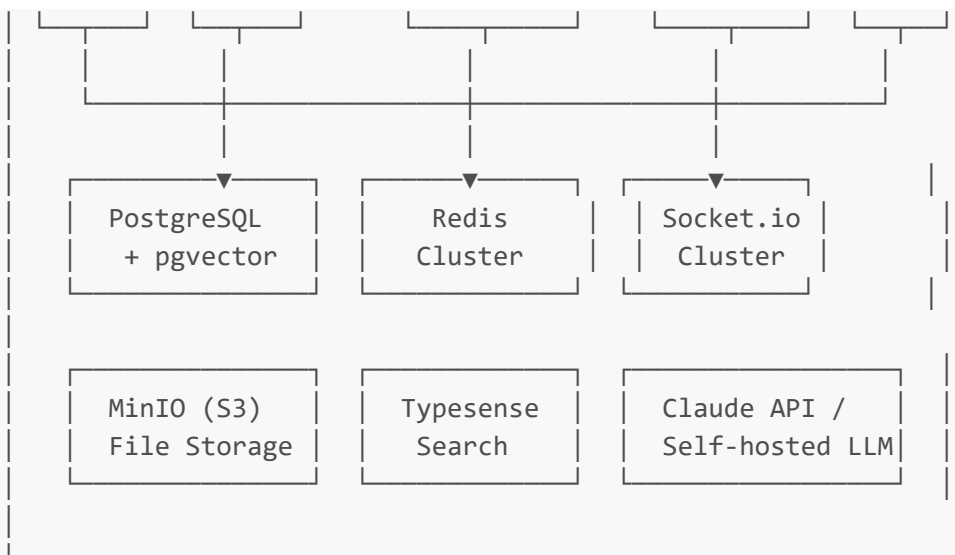
层级	技术	理由
向量数据库	pgvector / Qdrant	语义搜索、AI 总结
缓存	Redis	会话、实时数据
搜索引擎	Typesense / Meilisearch	全文搜索
对象存储	S3-compatible (MinIO)	论文、文件存储
队列	BullMQ	异步任务、AI 处理
实时	Socket.io	Tea Party
认证	NextAuth.js / Auth.js	OAuth + 邮箱登录

2.3 AI/ML 服务

功能	技术方案
信源总结	Claude API (长上下文)
质量评分	微调模型 / Claude Prompt Engineering
语义搜索	Embedding + pgvector
推荐系统	协同过滤 + 内容推荐
茶话会 AI 助手	Claude API + RAG

2.4 系统架构图





三、数据模型

3.1 核心实体关系

```

User
├── belongs to — ResearchGroup (as member)
├── follows — ResearchGroup
├── posts — Post (in discipline forum)
├── comments — Comment
├── votes — Vote (on posts/comments/publications)
└── participates — TeaPartyRoom

ResearchGroup
├── belongs to — Institution — College — Department
├── has — Publication[]
├── has — News[]
├── has — Patent[]
└── has — ScoreHistory

Discipline
├── has — SubDiscipline
├── topGroups — ResearchGroup[] (ranked by score)
└── posts — Post[]

Post
├── belongs to — Discipline (via tags)
├── has — Comment[]
├── has — Vote[]
└── linked — Publication[] (referenced papers)

Publication
├── authored_by — ResearchGroup[]
├── cited_in — Post[] (referenced in discussions)
└── summarized_by — AI_Summary

TeaPartyRoom
├── host — User
├── participants — User[]
└── messages — Message[]
  
```

3.2 评分权重系统

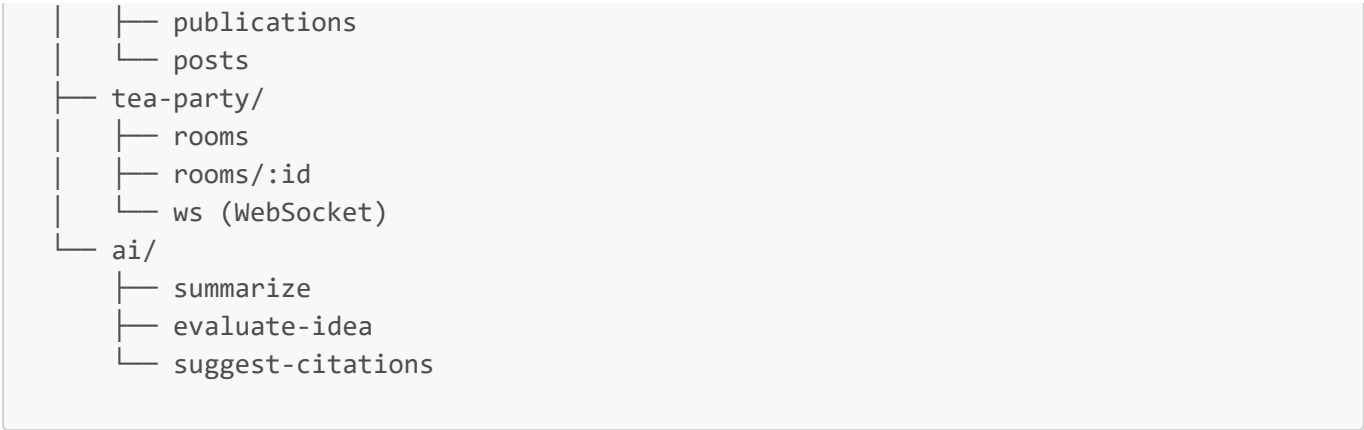
```
// 评分计算公式
const calculateGroupScore = (group: ResearchGroup) => {
  const publicationScore = publications.length * 10;
  const citationScore = citations * 5;
  const qualityCommentScore = qualityComments * 20; // 被引用或有启发性
  const normalCommentScore = normalComments * 5;
  const voteScore = votes * 1;

  return publicationScore + citationScore + qualityCommentScore +
    normalCommentScore + voteScore;
};
```

四、API 设计

4.1 RESTful API 结构

```
/api/v1/
├── auth/
│   ├── login
│   ├── register
│   ├── logout
│   └── oauth/:provider
├── users/
│   ├── profile
│   ├── publications
│   ├── groups
│   └── settings
├── groups/
│   ├── :id
│   ├── :id/members
│   ├── :id/publications
│   ├── :id/news
│   └── :id/score
├── disciplines/
│   ├── tree
│   ├── :id/posts
│   └── :id/top-groups
├── posts/
│   ├── :id
│   ├── :id/comments
│   └── :id/vote
├── publications/
│   ├── :id
│   ├── :id/summary
│   └── :id/citations
├── search/
│   └── groups
```



4.2 WebSocket 事件

```
// Tea Party Room Events
interface TeaPartyEvents {
  'room:join': { roomId: string; userId: string };
  'room:leave': { roomId: string; userId: string };
  'message:send': { roomId: string; content: string; type: 'text' | 'file' };
  'message:receive': { from: User; content: string; timestamp: Date };
  'screen:share:start': { roomId: string; streamId: string };
  'screen:share:stop': { roomId: string };
  'user:typing': { roomId: string; userId: string };
}
```

五、页面结构

5.1 页面清单

页面	路由	描述
首页	/	入口选择：高校/学科/TOP10/Tea Party
高校入口	/institutions	按高校浏览课题组
课题组主页	/groups/:id	课题组详情
学科入口	/disciplines	学科树导航
学科详情	/disciplines/:id	学科下课题组排行
板块讨论	/disciplines/:id/posts	讨论帖子列表
帖子详情	/posts/:id	帖子+评论
TOP10	/top-questions	本月热门问题
Tea Party	/tea-party	聊天室列表
茶话会房间	/tea-party/:id	实时聊天
思想工坊	/workshop	AI 对话

页面	路由	描述
用户主页	/users/:id	个人主页
搜索	/search	全局搜索

5.2 响应式设计

- **Desktop (≥1280px):** 完整三栏布局
- **Tablet (768-1279px):** 两栏布局，sidebar 可折叠
- **Mobile (<768px):** 单栏，底部导航栏

六、挑战与解决方案

6.1 数据冷启动

问题：初始缺乏课题组和论文数据

方案：

1. 与 Semantic Scholar API 合作获取公开论文数据
2. 开发数据导入工具，支持课题组批量上传
3. 设计激励机制鼓励用户完善信息
4. 邀请种子用户（知名课题组）入驻

6.2 AI 能力集成

问题：如何高效整合大模型能力

方案：

1. 统一 AI Service 层，封装多模型调用
2. 使用 LangChain 实现 RAG 流水线
3. 异步处理 AI 请求，避免阻塞
4. 实现 token 用量监控和配额控制

6.3 实时通信

问题：Tea Party 功能的高并发和低延迟

方案：

1. Socket.io 集群部署 + Redis Adapter
2. 房间状态分布式存储
3. 消息历史异步持久化
4. 降级策略：高峰期切换为异步模式

6.4 内容审核

问题：学术社区需要防止低质/不当内容

方案：

1. AI 辅助内容审核（第一道防线）
2. 用户举报 + 人工复核机制
3. 信任分系统：高信用用户免审
4. 敏感词过滤 + 学术诚信检测

6.5 跨学科知识图谱

问题： 如何建立学科间的关联

方案：

1. 多标签系统（一个帖子可属多个方向）
 2. 引用追踪图谱（展示知识流动）
 3. 相似研究方向推荐
 4. 跨学科合作匹配算法
-

七、迭代计划

Phase 1: MVP (3-4 个月)

- 用户注册/登录
- 课题组基础功能（CRUD）
- 信源上传 + AI 总结
- 基本评论/点赞

Phase 2: 社区 (2-3 个月)

- 学科社区
- 讨论帖子
- TOP10 投票
- 搜索功能

Phase 3: 实时 (2 个月)

- Tea Party 基本版
- 实时聊天
- 屏幕共享（可选）

Phase 4: AI 增强 (持续)

- 思想工坊
 - 智能推荐
 - 评分优化
 - 知识图谱
-

八、参考资源

开源项目参考

项目	用途	GitHub
Onyx	企业级 AI 搜索 + RAG 架构	onyx-dot-app/onyx (26.9k stars)
Hermes Agent	自学习 AI Agent	NousResearch/hermes-agent (74.6k stars)
oh-my-claudecode	Claude Code 多 Agent 编排	Yeachan-Heo/oh-my-claudecode (19 agents)
last30days-skill	热门讨论聚合	mvanhorn/last30days-skill (21.4k stars)
browser-use	浏览器自动化	browser-use/browser-use

设计参考

产品	参考点
GitHub	组织页面设计、代码/文档分离
cc98	中文论坛交互、话题聚合
Reddit	投票机制、评论区嵌套
Discord	实时聊天室、频道分类
Notion	文档编辑体验

九、风险评估

风险	概率	影响	缓解策略
数据冷启动失败	高	高	多渠道获取、种子用户激励
用户留存率低	中	高	差异化功能、快速迭代
AI 成本超支	中	中	配额控制、缓存策略
技术选型失误	低	高	充分调研、MVP 验证
竞争对手模仿	中	中	社区壁垒、数据积累

十、团队建议

核心能力需求

角色	数量	核心技能
全栈工程师	2-3	Next.js, Node.js, PostgreSQL
AI/ML 工程师	1	LLM, RAG, 向量数据库
UI/UX 设计师	1	Figma, 用户研究
产品经理	1	需求分析, 优先级

开发工具链

环节	工具
版本控制	Git + GitHub
CI/CD	GitHub Actions
监控	Grafana + Prometheus
日志	ELK Stack
文档	GitBook / Notion
设计	Figma
项目管理	Linear / Notion